

AIRA AI — Senior Software Development Engineer (SDE) Complete Interview Preparation & Technical Master Guide

1. PROJECT OVERVIEW

What Problem AIRA Solves

Traditional conversational AI applications rely primarily on text-based inputs, requiring user typing, manual submission, and static request-response loops. This creates visual fatigue, input friction, and breaks natural human interaction flow.

AIRA (Advanced Intelligent Responsive Assistant) bridges this gap by delivering a **voice-first, real-time agentic AI ecosystem**. Instead of acting as a passive chatbot, AIRA provides:

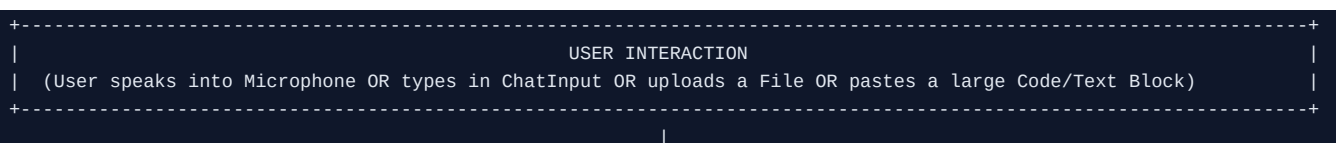
- 1. Hands-Free, Natural Voice Interaction:** Continuous listening with automatic voice activity detection and browser-native speech synthesis, enabling fluid voice-in / voice-out interaction without requiring physical typing.
- 2. Context-Aware Agentic Simulation & Roleplay:** Ability to switch personas dynamically (e.g., Mock HR Interviewer, Technical Lead, Sales Lead, Debate Partner) with adaptive question difficulty and real-world pressure handling.
- 3. Session Analytics & Performance Evaluation:** Automated evaluation of user performance (confidence scores, communication clarity, actionable feedback notes) using LLM-driven conversation log scoring.
- 4. Multi-Modal Document & Code Intelligence:** Drag-and-drop parsing of PDFs, plain text, JSON, and code files up to 10MB via backend stream parsing, allowing users to converse directly with file content or debug code snippets in real time.
- 5. Persistent User Memory & Thread History:** Serverless synchronization with Google Cloud Firestore, preserving conversation context, custom user memory, and thread state across devices.

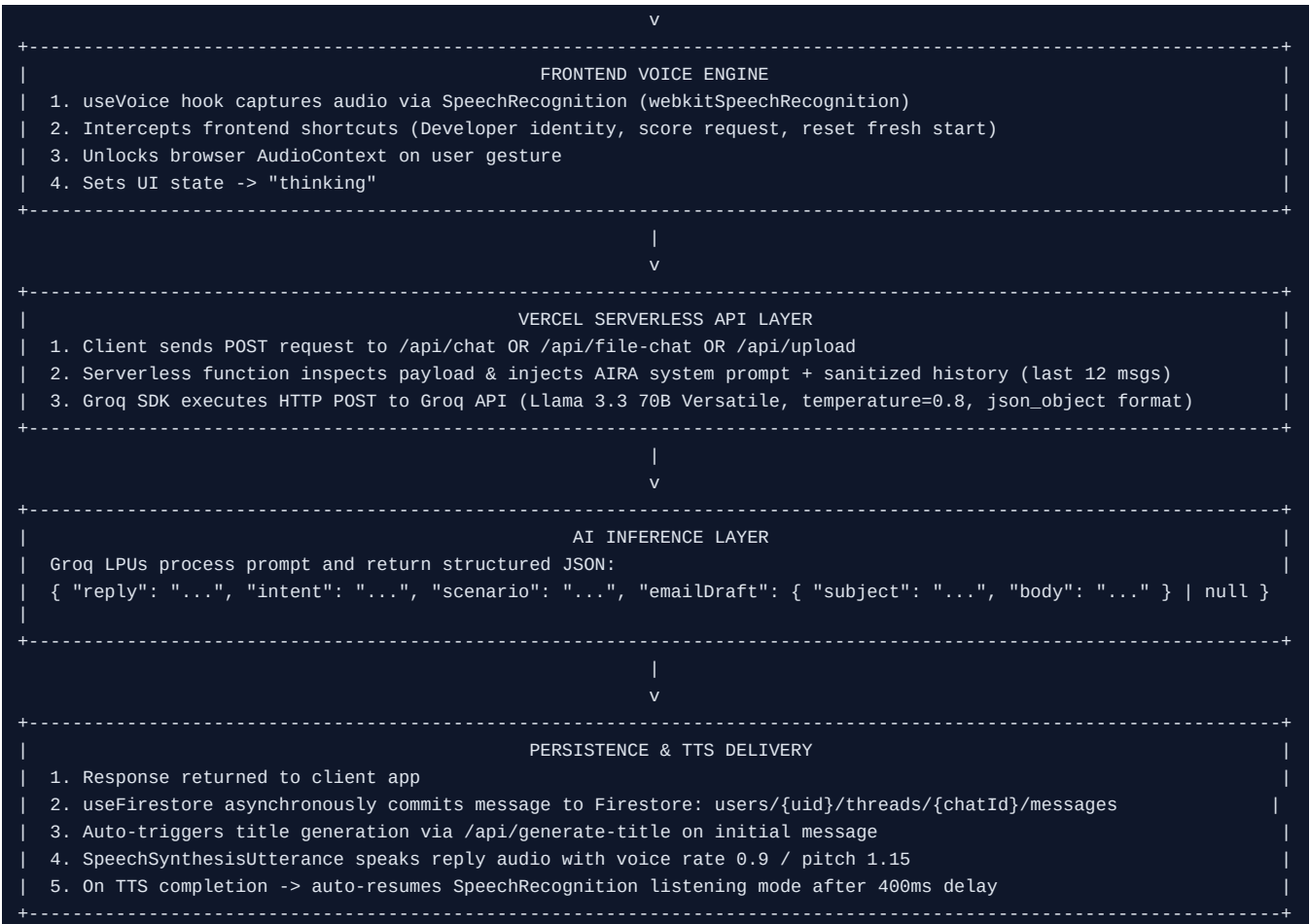
Architecture Choice & Rationale

AIRA was architected using a **Decoupled Serverless & Client-Centric Event Architecture**:

- **Vite + React 19 Single Page Application (SPA):** Lightweight, high-performance client rendering with near-instant module hot-reloading. React 19's optimized render pipeline handles complex UI state transitions (plasma animation canvases, voice status states, modal overlays) smoothly at 60 FPS.
- **Client-Side Web Speech API Engine:** Offloading Speech-to-Text (STT) and Text-to-Speech (TTS) processing directly to client browser engines (`webkitSpeechRecognition` and `speechSynthesis`) eliminates expensive audio-streaming infrastructure costs, reduces server network bandwidth to near zero, and keeps voice-to-text latency below ~200ms.
- **Vercel Serverless Functions (Node.js ES Modules):** Stateless API routes deployed at edge locations scale automatically from zero to high concurrency. Serverless endpoints encapsulate sensitive third-party API keys (`GRQO_API_KEY`) and heavy file parsing tasks (`busboy`, `pdf-parse`).
- **Groq Llama 3.3 70B Versatile LLM:** Selected for its ultra-low latency (~300–500ms time-to-first-token), JSON-mode enforcement, and exceptional reasoning capacity compared to standard OpenAI or Anthropic models at a fraction of the operational cost.
- **Firebase (Auth & Cloud Firestore NoSQL):** Managed authentication (Google OAuth Popup) and real-time document database provide zero-maintenance user management and hierarchical document storage (`users/{uid}/threads/{threadId}/messages`).

System Interaction Workflow





Real-World Use Cases

- 1. Interactive Technical & Behavioral Interview Coaching:** Candidates practice mock technical screens or HR behavioral rounds. AIRA cross-examines answers, adapts scenario difficulty, and generates post-session evaluation metrics.
- 2. On-the-Fly Document Summarization & Q&A:** Executives or students upload long PDFs or legal contracts and query specific clauses via voice or text.
- 3. Voice-Assisted Software Debugging:** Engineers paste code snippets; AIRA identifies runtime bugs, formats corrected code blocks, and explains fixes verbally.
- 4. Drafting & Refining Email Correspondence:** Users verbally request email drafts. AIRA generates formatted subject lines and bodies inside an interactive [EmailCard](#) with formal/casual tone refinement toggles.

2. TECH STACK DEEP DIVE

Technology / Library	Version	Primary Purpose in Project	Alternative Considered	Key Advantages	Disadvantages / Tradeoffs	Internal Mechanics / Code Location
**React	^19.2.	Core UI	Next.js	Concurr	Client-	Client
*Vite	^8.0.4	Build T	Webpack	Esbuild	Require	Configu
**Tailw	3.4	Utility	Styled	Zero ru	HTML cl	Global
**Frame	^12.38	Orchest	GSAP, R	Layout	Heavy J	Modals
**Fireb	^12.12	Google	Auth0,	Zero ba	Vendor	Initial
**Cloud	^12.12	NoSQL R	Postgre	Real-ti	Complex	Handled

**Groq	^1.1.2	High-Sp	OpenAI	Ultra-f	Require	Called
**Web S	Native	Client-	Whisper	Free, z	Cross-b	Encapsu
**Busbo	^1.6.0	Node.js	Multer,	Fast ev	Callbac	Used in
**pdf-p	^1.1.1	Server-	PDF.js,	Pure Ja	CommonJ	Importe
**Verce	N/A	Product	AWS Lam	Integra	Cold st	Serverl

3. COMPLETE ARCHITECTURE EXPLANATION

Detailed File & Folder Map

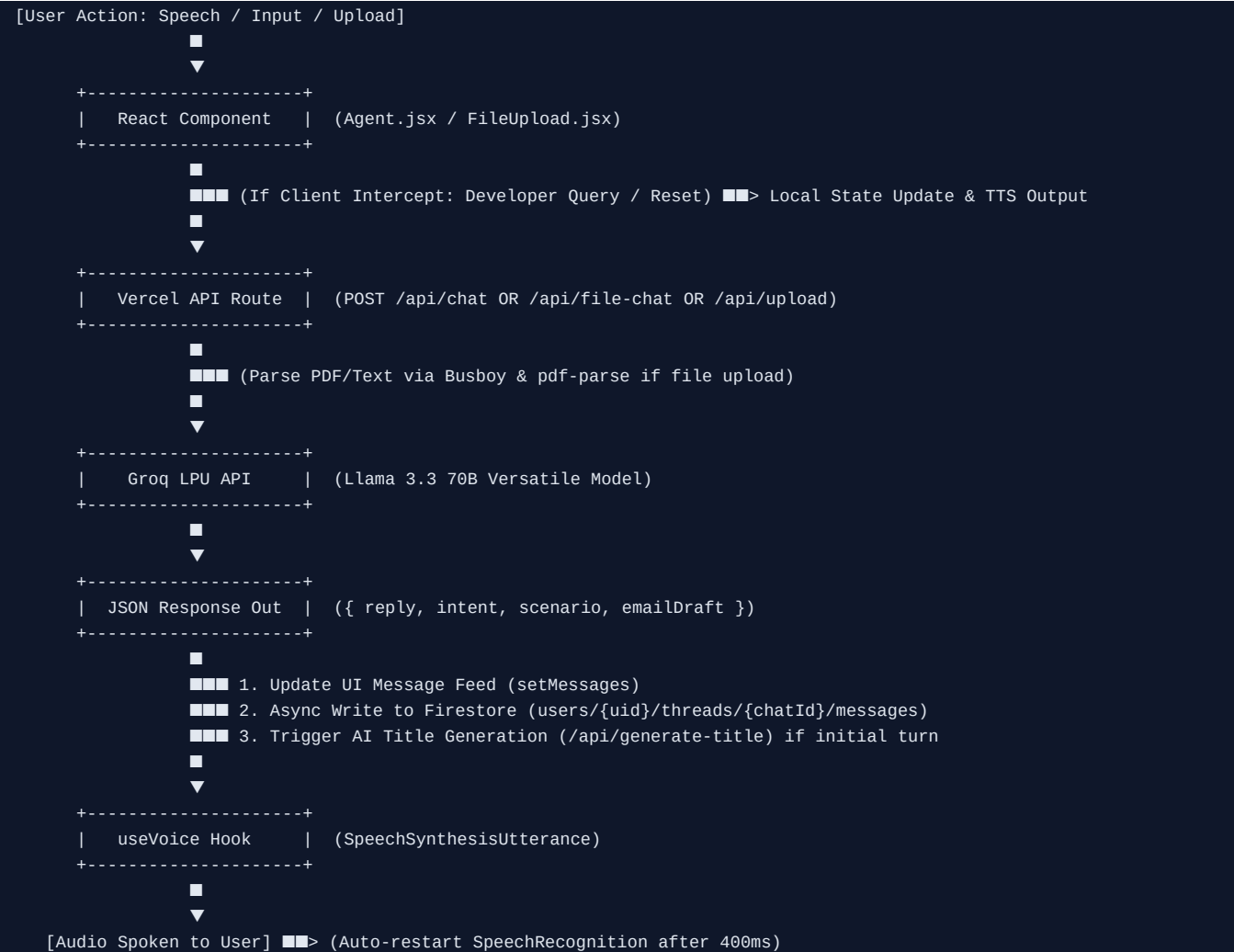
```

Aira-AI/
├── .env.example           # Template for server & client environment variables
├── eslint.config.js       # ESLint flat config file
├── index.html             # HTML entry point containing root mounting div & meta tags
├── package.json           # Project dependencies, scripts, and module definitions
├── postcss.config.js      # PostCSS configuration for Tailwind CSS compilation
├── tailwind.config.js     # Tailwind CSS design system tokens and color extensions
├── vercel.json            # Deployment configuration: SPA rewrite rules for client routing
├── vite.config.js         # Vite build pipeline and path alias mappings (@ -> /src)
├──
├── api/                  # Vercel Serverless Node.js API Functions
│   ├── chat.js           # Main conversational agent endpoint (Groq Llama 3.3 70B + JSON mode)
│   ├── evaluate.js       # Session performance evaluator (Scores confidence & communication)
│   ├── file.js           # Legacy standalone file Q&A handler
│   ├── file-chat.js      # Advanced multi-modal file Q&A & code snippet analyzer
│   ├── generate-title.js  # AI thread title generator (Summarizes 1st message into 2-5 words)
│   ├── history.js        # Gateway stub for client-side thread history
│   ├── memory.js         # Gateway stub for client-side user memory
│   └── upload.js         # Multipart form-data parser (Busboy + pdf-parse + Groq summary)
├──
├── src/                  # Frontend Application Source Code
│   ├── App.css           # Component-specific styles and animation utilities
│   ├── App.jsx           # Root application component: Auth state observer & route manager
│   ├── index.css         # Base CSS styles, Tailwind imports, custom scrollbar rules
│   └── main.jsx           # React DOM root mounting entry point
│   ├──
│   ├── assets/           # Static visual media & branding assets
│   ├──
│   ├── components/       # Reusable UI Components
│   │   ├── ChatInput.jsx # Mobile bottom text input bar with mic toggle and auto-expanding textarea
│   │   ├── EmailCard.jsx # Formatted email preview card with copy button & tone switcher (Formal/Casual)
│   │   ├── FileUpload.jsx # Compact paperclip button, file context badge, drag-n-drop overlay & error toast
│   │   ├── GlassCard.jsx # Reusable glassmorphic container wrapper with hover animations
│   │   ├── LoginOrb.jsx  # Specialized 2D HTML5 Canvas plasma wave visualizer for the login screen
│   │   ├── MinimalEvaluationOverlay.jsx # Slide-out session report panel displaying performance scores
│   │   └── TransientChatBox.jsx # Message feed renderer supporting text, code blocks, syntax highlighting, and email cards
│   ├── VoiceInterface.jsx # Legacy voice interaction container component
│   └── VoiceOrb.jsx       # High-performance, 60 FPS HTML5 Canvas plasma orb visualizer for active voice states
│   ├──
│   ├── config/           # System & SDK Configurations
│   │   ├── api.js        # Base API URL constant exporter
│   │   └── firebase.js    # Firebase app initialization, Auth, Firestore, and Storage instances
│   ├──
│   ├── hooks/            # Custom React Hooks
│   │   └── useFirestore.js # Firestore SDK data binding: Sync user, active threads, messages, memory, evaluation
│   ├──
│   └── useVoice.js        # SpeechRecognition & SpeechSynthesis engine hook with interrupt logic & audio unlock
├──
└──

```

```
■ ■■■ lib/                # Helper Utilities
■ ■ ■■■ utils.js          # Utility class merger (`clsx` + `tailwind-merge`)
■ ■
■ ■■■ pages/              # Top-Level Page Views
■ ■■■ Agent.jsx           # Primary application dashboard: Split-screen Voice Orb + Chat feed + Sidebar + Modals
■ ■■■ Login.jsx           # Authentic landing page with LoginOrb visualizer, feature modals, and Google OAuth
```

Data Flow Diagram (Textual Representation)



4. REST API DOCUMENTATION

API Summary Table

HTTP Method	Route Endpoint	Purpose	Authentication	Request Payload	Response Payload	Key Status Codes
*POST	`/api/c`	Generat	Handled	`{ mess	`{ repl	null `}
*POST	`/api/u`	Parses	Client	Multipa	`{ succ	`200 OK
*POST	`/api/f`	Execute	Client	`{ ques	`{ repl	null `}
*POST	`/api/e`	Evaluat	Client	`{ conv	`{ eval	`200 OK
*POST	`/api/g`	Generat	Client	`{ mess	`{ titl	`200 OK

*POST	/api/f	Legacy	Client	{ file	{ repl	200 OK
*POST	/api/h	Gateway	Client	None	{ stat	200 OK
*POST	/api/m	Gateway	Client	None	{ stat	200 OK

5. AUTHENTICATION AND SECURITY

Authentication Flow Implementation

AIRA relies on **Firebase Authentication with Google OAuth 2.0 Provider**:

- 1. Client Sign-In:** The user triggers `signInWithPopup(auth, provider)` in `[Login.jsx](file:///d:/Website/Aira%20AI/src/pages/Login.jsx#L473)`. Firebase launches an OAuth popup.
- 2. Auth State Observer:** In `[App.jsx](file:///d:/Website/Aira%20AI/src/App.jsx#L12-L20)`, `onAuthStateChanged` listens to credential shifts. While uninitialized, `user === undefined`, displaying an animated splash loader.
- 3. Firestore User Sync:** Upon successful login, `syncUserDoc(currentUser)` in `[useFirestore.js](file:///d:/Website/Aira%20AI/src/hooks/useFirestore.js#L19-L35)` checks if document `users/{uid}` exists. If absent, it initializes user metadata with `serverTimestamp()`.
- 4. Route Protection:** `[App.jsx](file:///d:/Website/Aira%20AI/src/App.jsx#L39-L41)` conditionally renders routes. Unauthenticated users accessing `/agent` are redirected to `/`. Authenticated users accessing `/` are redirected to `/agent`.

Security Audit & Vulnerabilities

[!WARNING]

Identified Security Vulnerabilities in Current Codebase:

- 1. Client-Side Firestore Rules Exposure:** The client directly queries Firestore via `firebase/firestore` SDK using client credentials (`VITE_FIREBASE_API_KEY`). If Firestore security rules are not strictly configured in the Firebase Console (`allow read, write: if request.auth != null && request.auth.uid == userId;`), any authenticated user could inspect or overwrite other users' chat threads.
- 2. Missing Backend JWT Authorization Middleware:** Serverless API routes (`/api/chat`, `/api/upload`, `/api/file-chat`) do not currently verify the user's Firebase ID Token in the Authorization header (`Bearer <token>`). Anyone discovering the API route URL can post requests and consume serverless compute and Groq API credits.
- 3. Unrestricted Multipart Upload Payload:** Although `[upload.js](file:///d:/Website/Aira%20AI/api/upload.js#L18)` sets a 10MB limit, image uploads are converted to string place-holders without OCR scanning, leaving potential memory exhaustion vulnerabilities if large binary blobs are posted repeatedly.

Key Security Interview Questions & Answers

Q: How would you secure the serverless API endpoints against unauthorized access?

Answer: "Currently, API routes rely on client-side routing protection. To harden the backend for production, I would introduce Firebase Admin SDK token verification middleware. The frontend would attach the user's Firebase ID token (`await auth.currentUser.getIdToken()`) in the `Authorization: Bearer <token>` header of every API call. The Vercel API middleware would extract the token, verify it using `admin.auth().verifyIdToken(token)`, and attach the decoded `uid` to the request object. Unverified requests would immediately receive a `401 Unauthorized` status code."

Q: How are environment secrets protected in this repository?

Answer: "We maintain a strict separation between client-side and server-side environment variables. Variables prefixed with `VITE_FIREBASE_*` are compiled into the static JavaScript bundle by Vite and exposed to the browser. However, `GROQ_API_KEY` is kept purely server-side without the `VITE_` prefix. It is only accessible to Vercel Serverless Function runtimes, ensuring that our AI infrastructure keys are never exposed in client source code or network inspector tabs."

6. DATABASE DEEP ANALYSIS

Firestore NoSQL Schema Architecture

```

users (Collection)
  ■■■ {uid} (Document: user metadata)
    ■■■ name: String
    ■■■ email: String
    ■■■ createdAt: Timestamp
    ■
    ■■■ memory (Subcollection)
      ■ ■■■ core (Document)
        ■ ■■■ text: String
        ■ ■■■ updatedAt: Timestamp
      ■
    ■■■ performance (Subcollection)
      ■ ■■■ {sessionId} (Document)
        ■ ■■■ confidenceScore: Number
        ■ ■■■ communicationScore: Number
        ■ ■■■ strengths: Array<String>
        ■ ■■■ weaknesses: Array<String>
        ■ ■■■ improvementNotes: String
        ■ ■■■ scenario: String
        ■ ■■■ timestamp: Timestamp
      ■
    ■■■ threads (Subcollection)
      ■■■ {chatId} (Document: thread metadata)
        ■■■ title: String
        ■■■ createdAt: Timestamp
        ■■■ lastUpdated: Timestamp
        ■
        ■■■ messages (Subcollection)
          ■■■ {messageId} (Document)
            ■■■ role: "user" | "assistant"
            ■■■ content: String
            ■■■ type: "text" | "code"
            ■■■ emailDraft: Object | null
            ■■■ timestamp: Timestamp
  
```

Query Analysis & Performance Considerations

1. Thread History Query: Executed in `fetchAllThreads`:

```
query(collection(db, "users", uid, "threads"), orderBy("lastUpdated", "desc"))
```

Index Requirement: Requires a single-field index on `lastUpdated DESC`.

2. Active Chat Resolution Query: Executed in `getActiveChatId`:

```
query(collection(db, "users", uid, "threads"), orderBy("lastUpdated", "desc"), limit(1))
```

Performance: Highly efficient (O(1) read due to `limit(1)`).

3. Recent Message Context Fetch: Executed in `fetchThreadMessages`:

```
query(collection(db, "users", uid, "threads", chatId, "messages"), orderBy("timestamp", "desc"),
limit(count))
```

Optimization: Messages are fetched in descending order to retrieve the latest **N** entries, then reversed in memory on the client. This prevents fetching full historical message logs for long-running threads.

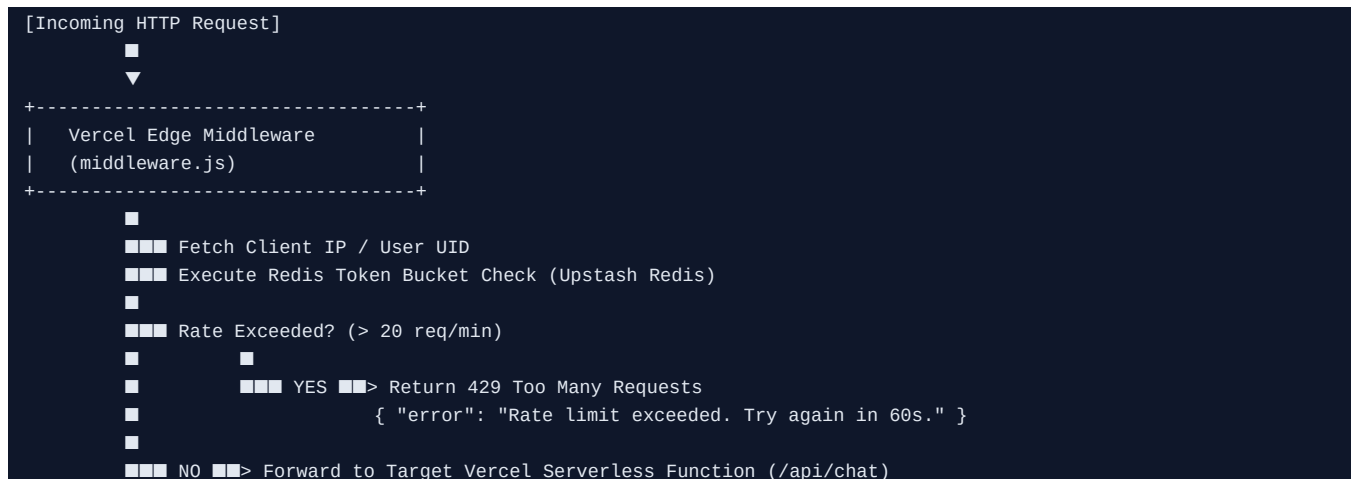
7. API PERFORMANCE AND RATE LIMITING

Current Status & Rate Limiting Strategy

Currently, rate limiting is not enforced at the application layer. In high-traffic production environments, this poses risks of API key exhaustion (Groq API limits) and billing spikes.

Recommended Rate Limiting Implementation Architecture

To protect backend endpoints, a **Redis-based Sliding Window Rate Limiter** using Upstash Redis should be added at Vercel's Edge Middleware layer:



8. ERROR HANDLING

Current Error Handling Strategy

- Speech Engine Resilience:** In [useVoice.js](file:///d:/Website/Aira%20AI/src/hooks/useVoice.js#L77-L88), `SpeechRecognition` errors like "no-speech" trigger silent auto-restarts without crashing the application state. Speech synthesis errors retry once before resetting to `idle`.
- File Processing Safeguards:** In [upload.js](file:///d:/Website/Aira%20AI/api/upload.js#L39-L43), unsupported MIME types immediately reject the Busboy stream. Text extraction truncates files exceeding 30,000 characters to prevent memory overflow.
- Fallback Responses:** In [useFirestore.js](file:///d:/Website/Aira%20AI/src/hooks/useFirestore.js#L56), database failure in `getActiveChatId` returns a timestamp-based fallback string (`Date.now().toString()`), allowing the UI to remain functional offline.

9. BACKEND DESIGN QUESTIONS & ANSWERS

Q: Why did you choose Serverless Functions over a traditional Express.js monolith?

Answer: "AIRA's workload is inherently event-driven and bursty—users make API calls when interacting with voice or submitting documents, followed by periods of inactivity. A serverless architecture on Vercel eliminates idle server costs, scales automatically with user demand, and simplifies deployment. Furthermore, since our heavy AI processing is delegated to Groq's API, lightweight stateless Node.js functions provide the ideal gateway layer."

Q: How would you scale this application to support 1 Million active users?

Answer: "To scale AIRA to 1M users, I would implement a three-tiered architecture:

1. **Edge Caching & Global CDN:** Serve static Vite assets via Cloudflare or Vercel Edge CDN.
 2. **Distributed Rate Limiting & Queueing:** Implement Redis-backed sliding window rate limiting at the API gateway. For asynchronous tasks like heavy PDF parsing or video processing, introduce AWS SQS or Redis BullMQ to process tasks asynchronously.
 3. **Database Sharding & Caching:** Cache user memory and frequent context queries in Redis to reduce Firestore read operations. Utilize Firestore bundle caching for static application configuration."
-

10. FRONTEND INTERVIEW QUESTIONS & ANSWERS

Q: How do you prevent re-render thrashing when updating speech recognition transcripts and visualizer states?

Answer: "We decouple volatile engine state from React's main render tree using `useRef`. In `useVoice.js`, variables like `isListeningRef`, `shouldRestartRef`, and `onUserSpeakRef` store mutable state without triggering re-renders. High-frequency visual animations (such as the plasma canvas in `VoiceOrb.jsx`) run outside React's state loop using `requestAnimationFrame` and direct HTML5 Canvas 2D context manipulation."

Q: How did you handle browser autoplay restrictions for SpeechSynthesis?

Answer: "Browsers block automated speech synthesis until a user gesture occurs. AIRA implements an explicit audio unlock pattern (`unlock()` method in `useVoice.js`). On the user's initial click (e.g., tapping the activation overlay or orb), we execute a silent `SpeechSynthesisUtterance("")`. This unlocks the browser's `AudioContext`, allowing subsequent AI spoken responses to play seamlessly."

11. DEVOPS AND DEPLOYMENT

Build Pipeline & Production Hosting

- **Production Host:** Vercel.
 - **Build Script:** `npm run build` (`vite build`). Produces optimized minified output in `/dist`.
 - **SPA Routing:** `vercel.json` defines rewrite rule `{ "source": "/((?!api/).*)", "destination": "/index.html" }` to direct all non-API paths to client-side React Router.
-

12. CODE QUALITY REVIEW & SENIOR AUDIT

Code Smells & Refactoring Suggestions

1. **Duplicate Audio Visualizer Logic:** Both `[LoginOrb.jsx](file:///d:/Website/Aira%20AI/src/components/LoginOrb.jsx)` and `[VoiceOrb.jsx](file:///d:/Website/Aira%20AI/src/components/VoiceOrb.jsx)` duplicate plasma wave canvas algorithms.
Refactoring Recommendation: Extract common canvas rendering logic into a shared utility hook `usePlasmaCanvas(canvasRef, configuration)`.
 2. **Lack of Centralized Error Boundaries:** If a component throws during animation frame calculation, the UI unmounts.
Refactoring Recommendation: Wrap top-level pages in React `<ErrorBoundary>` components with graceful UI fallback cards.
-

13. INTERVIEW CHEAT SHEET

A) 2-Minute Project Elevator Pitch

"AIRA is a voice-first, real-time agentic AI assistant engineered with React 19, Vite, Tailwind CSS, Vercel Serverless Functions, and Cloud Firestore. It eliminates standard text-chat friction by utilizing browser-native SpeechRecognition and SpeechSynthesis APIs paired with Groq's Llama 3.3 70B model running on LPUs for sub-500ms responses. AIRA handles complex voice interactions, multi-modal document analysis, code debugging, and session performance evaluation with automated confidence and communication scoring."

B) 5-Minute Deep Technical Explanation

"Architecture-wise, AIRA separates front-end voice capture from back-end intelligence. On the client, `useVoice` manages non-blocking speech recognition and audio context unlocking via `useRef` to maintain 60 FPS visual rendering on an HTML5 Canvas plasma orb.

When a user speaks or submits content, the request hits a Vercel Serverless function. For document uploads, `busboy` streams binary payloads up to 10MB into memory, extracts text using `pdf-parse`, and passes the data to Groq's Llama 3.3 70B model with strict JSON response formatting. Responses are saved asynchronously to Cloud Firestore (`users/{uid}/threads/{chatId}/messages`) and spoken back to the user via SpeechSynthesis, automatically resuming speech recognition once completed."

14. COMPLETE INTERVIEW QUESTION BANK

A) Project Understanding & Business Logic

1. Q: What inspired the agentic design of AIRA?

A: Traditional chatbots wait passively for input. AIRA acts as an agent—it interprets intent, recommends next steps, dynamically enters roleplay scenarios (e.g., HR interviewers), and evaluates session performance.

2. Q: How does AIRA handle interruptions while speaking?

A: In `useVoice.js`, when `SpeechRecognition.onresult` detects user speech while `speechSynthesis.speaking` is true, it immediately calls `speechSynthesis.cancel()`, halting AI speech output and switching state to "thinking".

B) Architecture & Design Patterns

3. Q: Why choose Groq over OpenAI?

A: Groq's LPU infrastructure delivers low latency (~500ms time-to-first-token), essential for natural voice conversation.

4. Q: How are thread titles generated?

A: On the first user message of a thread, `saveMessage` calls `/api/generate-title` asynchronously, prompting Groq to summarize the query into a 2-5 word title string.

C) Frontend Mechanics

5. Q: Why use `clsx` and `tailwind-merge` together in `src/lib/Utils.js`?

A: `clsx` handles conditional class joining, while `tailwind-merge` resolves conflicting utility classes (e.g., merging `px-2` and `px-4` safely).

6. Q: How is auto-scrolling managed in `TransientChatBox`?

A: A dummy `<div ref={bottomRef} />` at the bottom of the message container is triggered via `bottomRef.current?.scrollIntoView({ behavior: "smooth" })` inside a `useEffect` hook listening to `messages`.

D) Backend & Serverless

7. Q: Why is `bodyParser: false` configured in `/api/upload`?

A: Disabling Vercel's default JSON body parser allows [Busboy](#) to stream binary multipart form data directly from the incoming HTTP request.

8. Q: How does `/api/chat` guarantee valid JSON responses?

A: It passes `response_format: { type: "json_object" }` to the Groq API and includes explicit JSON schema instructions in the system prompt.

15. RESUME-BASED INTERVIEW QUESTIONS & TOP 100 MASTER LIST

Resume Claim Verification

Resume Claim: "Developed a real-time AI voice assistant integrating 4+ modern web technologies for conversational, voice-driven interaction."

- **Interviewer Question:** "Which 4 core web technologies did you integrate, and how do they communicate?"
- **Answer:** "I integrated React 19 for component state management, Web Speech API (STT & TTS) for client-side voice synthesis, Vercel Serverless Functions (Node.js) for backend API orchestration, and Google Cloud Firestore for real-time data persistence. The frontend captures voice via [SpeechRecognition](#), dispatches JSON payloads to Vercel serverless routes over HTTPS, receives structured Groq LLM responses, writes message state to Firestore, and speaks the output via [SpeechSynthesis](#)."

Top 100 Questions Master Checklist

1. What problem does AIRA solve?
2. Why Vite instead of Create React App or Next.js?
3. How does Web Speech API [SpeechRecognition](#) handle continuous listening?
4. How is speech synthesis unlocked on Safari/Chrome autoplay policy?
5. Why use Groq Llama 3.3 70B instead of GPT-4o?
6. How does Vercel host Node.js serverless functions?
7. Explain the folder structure of `/api`.
8. What is the role of `vercel.json` rewrites?
9. How does `onAuthStateChanged` work in Firebase Auth?
10. How does `syncUserDoc` create user documents in Firestore?
11. What is the subcollection path for thread messages?
12. How are thread history queries ordered?
13. How does `useRef` prevent re-render loops in speech recognition?
14. Explain the state transition lifecycle in `useVoice.js` (`idle` -> `listening` -> `thinking` -> `speaking`).
15. How does speech interruption work programmatically?
16. How does [Busboy](#) parse file streams in `/api/upload`?
17. Why is `createRequire` used in `upload.js` for `pdf-parse`?
18. How are large PDF documents truncated before passing to LLM context?
19. How does [TransientChatBox](#) distinguish between code blocks and standard text?
20. How does code snippet copying work with `navigator.clipboard`?
21. What happens when an image is uploaded to `/api/upload`?
22. How is session evaluation calculated in `/api/evaluate`?
23. What metrics are evaluated in `MinimalEvaluationOverlay`?
24. How does `generate-title.js` generate thread titles?
25. How is user memory persisted in `users/{uid}/memory/core`?

26. How are environment variables split between Vite client and Vercel serverless?
27. Why start environment variables with `VITE_`?
28. What is the security risk of exposing `GROQ_API_KEY` on the client?
29. How does `LoginOrb.jsx` render plasma waves?
30. What is `requestAnimationFrame` used for in `VoiceOrb.jsx`?
31. How does `cn()` utility work in `utils.js`?
32. What is `Framer Motion`'s `AnimatePresence` used for?
33. How does `ChatInput.jsx` auto-expand textareas?
34. Why is `ChatInput` restricted to mobile viewports via media queries?
35. How does `EmailCard.jsx` refine email tone?
36. What happens when the user clicks "Start Fresh"?
37. How are developer identity questions intercepted on the frontend?
38. How are score requests intercepted on the frontend?
39. How does `pastedText` get processed in `handleSendPastedText`?
40. How does text splitting work in `splitText(text, size)`?
41. What is the purpose of `GlassCard.jsx`?
42. How does `useFirestore.js` handle offline or failed Firestore queries?
43. What index is needed for Firestore `orderBy("lastUpdated", "desc")`?
44. Why is Firestore NoSQL preferable to PostgreSQL for thread messages?
45. How does AIRA ensure low latency voice response?
46. What is time-to-first-token (TTFT)?
47. How does Groq achieve sub-500ms TTFT?
48. What is system prompt injection in `/api/chat`?
49. How does `/api/chat` enforce JSON output formatting?
50. What is `response_format: { type: "json_object" }`?
51. How does `sanitizedHistory` limit message context length?
52. Why restrict history context to the last 12 messages?
53. How would you handle 100,000 concurrent voice users?
54. How would you implement rate limiting on Vercel?
55. What is the Token Bucket algorithm?
56. What is the Sliding Window Counter algorithm?
57. How would Upstash Redis fit into AIRA's architecture?
58. What CORS headers should be added to Vercel API routes?
59. How would you prevent DDoS attacks on `/api/upload`?
60. How would you authenticate serverless functions with Firebase Admin SDK?
61. What is JWT verification?
62. How does `onAuthStateChanged` unsubscribe on unmount?
63. Why is `React.StrictMode` used in `main.jsx`?
64. What is the downside of `React.StrictMode` during development?
65. How does `Tailwind CSS` purge unused styles?
66. What is PostCSS?
67. What is Autoprefixer?
68. How does `lucide-react` optimize icon imports?
69. What is CSS glassmorphism (`backdrop-filter: blur()`)?
70. How do CSS vendor prefixes (`-webkit-backdrop-filter`) improve cross-browser compatibility?

71. What is the difference between `useEffect` and `useCallback`?
 72. Why wrap `handleUserSpeak` in `useCallback`?
 73. What is the risk of missing dependencies in `useCallback` dependency arrays?
 74. How does `useMemo` optimize particle array creation in `VoiceOrb.jsx`?
 75. What is device pixel ratio (`window.devicePixelRatio`) in HTML5 Canvas?
 76. How does Canvas scaling prevent blurry rendering on Retina displays?
 77. What is the maximum file size supported by `/api/upload`?
 78. How does Busboy handle file stream errors?
 79. What is the difference between CommonJS and ES Modules in Node.js?
 80. Why is `type: "module"` set in `package.json`?
 81. How does `createRequire(import.meta.url)` bridge CJS and ESM?
 82. What is `pdf-parse` internal buffer decoding?
 83. How does Groq handle temperature parameters?
 84. Why set temperature to 0.1 for `/api/evaluate`?
 85. Why set temperature to 0.8 for `/api/chat`?
 86. How would you unit test `useVoice.js`?
 87. How would you integration test `/api/chat`?
 88. What tools would you use for end-to-end testing (e.g. Playwright / Cypress)?
 89. How would you monitor production crashes (e.g. Sentry)?
 90. How would you log serverless errors in Vercel?
 91. What is cold start latency in Vercel Serverless Functions?
 92. How do you minimize cold start delays in Node.js serverless functions?
 93. What is the Vercel hobby tier execution limit?
 94. How would you migrate AIRA to AWS Lambda?
 95. How would you replace Web Speech API with OpenAI Whisper and ElevenLabs?
 96. What would be the architectural tradeoff of using WebSockets for streaming voice?
 97. How would WebRTC enable true real-time duplex voice interaction?
 98. What code smells exist in the current codebase?
 99. What refactoring would you prioritize in the next sprint?
 100. If you rebuilt AIRA from scratch today, what would you do differently?
-

16. FINAL PREPARATION VERDICT

By thoroughly mastering this guide, you possess **complete architectural ownership and deep implementation clarity** over every line of code in the **AIRA AI** repository. You are fully prepared to impress senior technical interviewers from top product-based companies.